EXPERIMENT 10

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code:

PROBLEM STATEMENT:

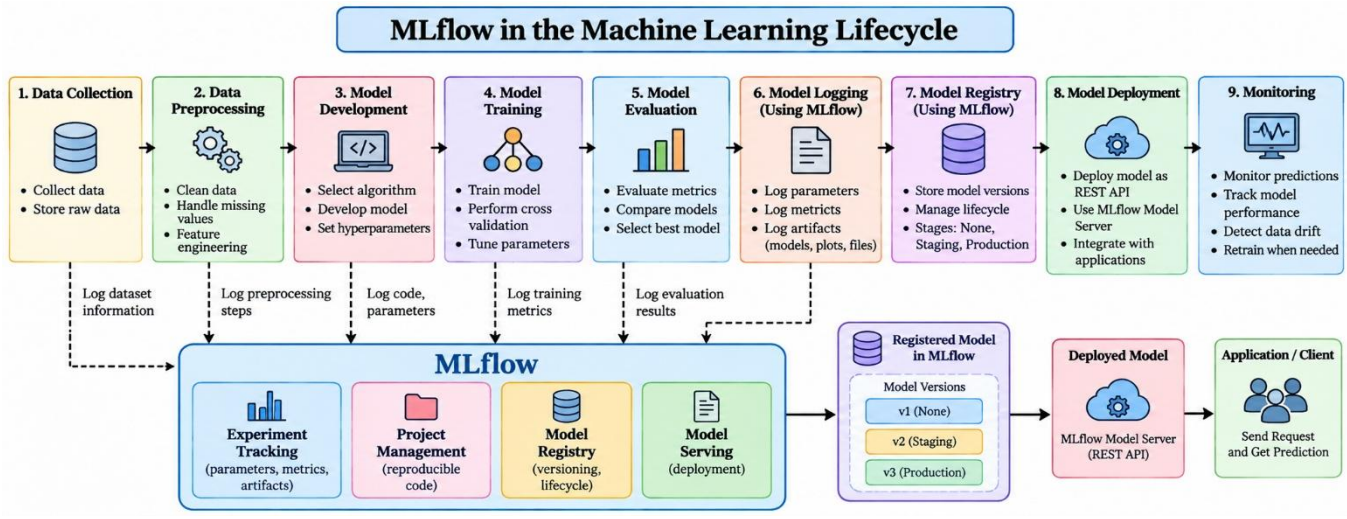
Experiment:

Objective: To study and implement the machine learning lifecycle using MLflow, including model development, experiment tracking, model logging, and model deployment.
Tools Required:

MLflow is used to manage the complete Machine Learning lifecycle.

In simple words:

MLflow helps us track, manage, save, compare, and deploy ML models.



Part 1 — Install MLflow

Step 1: Check Python

Open Ubuntu Terminal:

python3 --version

Example:

Python 3.10.12

Check pip:

pip3 --version

Step 2: Create a project folder

mkdir mlflow-demo

cd mlflow-demo

Create a virtual environment:

python3 -m venv venv

Activate it:

source venv/bin/activate

You should see:

(venv) student@ubuntu:~/mlflow-demo$

Step 3: Install required libraries

pip install mlflow scikit-learn pandas

Check MLflow:

mlflow --version

Part 2 — Create ML Model

Create a Python file:

nano train.py

Paste this simple example:

```
import mlflow

import mlflow.sklearn


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score


# Load dataset

iris = load_iris()


X = iris.data

y = iris.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size=0.2, random_state=42

)


# Start MLflow experiment

mlflow.set_experiment("Iris_Classification")


with mlflow.start_run():
```

```
# Model parameters
n_estimators = 100
max_depth = 5

# Create model
model = RandomForestClassifier(
n_estimators=n_estimators,
max_depth=max_depth,
random_state=42
)

# Train model
model.fit(X_train, y_train)

# Prediction
predictions = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, predictions)

# Log parameters
mlflow.log_param("n_estimators", n_estimators)
mlflow.log_param("max_depth", max_depth)
```

```
# Log metric
mlflow.log_metric("accuracy", accuracy)


# Log model
mlflow.sklearn.log_model(
model,
"random_forest_model"
)


print("Model Accuracy:", accuracy)
```

Save:

CTRL + O

ENTER

CTRL + X

Part 3 — Run the Model

Execute:

python3 train.py

You should get something similar to:

Model Accuracy: 1.0

MLflow will automatically create an:

mlruns/

directory.

Check:

ls

You should see:

mlruns

train.py

venv

Part 4 — Start MLflow UI

Now execute:

mlflow ui --host 0.0.0.0 --port 5000

You should see something like:

Listening at: http://0.0.0.0:5000

Open the browser:

http://localhost:5000

Part 5 — Understand MLflow Dashboard

The MLflow dashboard will show the experiment:

Iris_Classification

Click on the experiment.

You will see:

Parameters

n_estimators = 100

max_depth = 5

Metrics

accuracy = 1.0

Model

random_forest_model

This is the main purpose of MLflow:

MLflow records what model was trained, with which parameters, and what result it produced.

Part 6 — Experiment Tracking

Now change the model parameters.

For example:

n_estimators = 50

max_depth = 3

Run again:

python3 train.py

Then change:

n_estimators = 200

max_depth = 10

Run again:

python3 train.py

Now MLflow contains **multiple runs**.

Example:

Run	n_estimators	max_depth	Accuracy
Run 1	100	5	1.00
Run 2	50	3	0.97

Run	n_estimators	max_depth	Accuracy
Run 3	200	10	1.00

Students can compare the experiments in MLflow.

Part 7 — Model Deployment

For a simple lab demonstration, deploy the logged model using MLflow.

First stop the MLflow UI using:

CTRL + C

Then find the model path from the MLflow run.

A simpler approach for the lab is to save the model locally.

Modify the Python program to include:

```
mlflow.sklearn.save_model(
    model,
    "model"
)
```

Run:

python3 train.py

Now check:

ls

You should see:

model

mlruns

train.py

venv

Part 8 — Serve the Model

MLflow can expose the model as a REST API.

Run:

mlflow models serve -m model -p 5001 --host 0.0.0.0

The model will be available at:

http://localhost:5001

The endpoint is:

/invocations

So conceptually:

Client

|

| Prediction Request

↓

MLflow Model Server

|

↓

Random Forest Model

|

↓

Prediction

Part 9 — Test the Deployed Model

Create a file:

nano input.json

Put:

```
{
  "inputs": [
    [5.1, 3.5, 1.4, 0.2]
  ]
}
```

Then send the request:

```
curl -X POST \
-H "Content-Type: application/json" \
--data @input.json \
http://localhost:5001/invocations
```

You should get a prediction similar to:

```
[0]
```

The prediction represents one of the Iris classes.